Slide_1

Why relational database: A **relational database** is a database that has a collection of tables of data items, all of which is formally described and organized according to the relational model. Data in a single table represents a relation, from which the name of the database type comes. In typical solutions, tables may have additionally defined relationships with each other.
1.We all have the built in understanding of logic in its simplest form. 2. Relational concept with its reference to simplest data structure are very intuitive. 3. The declarative description of queries can be easily compiled into procedural description via relational algebra. 4. It is easy to explain to the people of office the basic relational operations.
Why indexing needed: When data is stored on disk based storage devices, it is stored as blocks of data. These blocks are accessed in their entirety, making them the atomic disk access operation. Disk blocks are structured in much the same way as linked lists; both contain a section for data, a pointer to the location of the next node (or block), and both need not be stored contiguously.Due to the fact that a number of records can only be sorted on one field, we can state that searching on a field that isn't sorted requires a Linear Search which requires N/2 block accesses (on average), where N is the number of blocks that the table spans. If that field is a non-key field (i.e. doesn't contain unique entries) then the entire table space must be searched at N block accesses.Whereas with a sorted field, a Binary Search may be used, this has log2 N block accesses. Also since the data is sorted given a non-key field, the rest of the table doesn't need to be searched for duplicate values, once a higher value is found. Thus the performance increase is substantial. **What is indexing?**Indexing is a way of sorting a number of records on multiple fields. Creating an index on a field in a table creates another data structure which holds the field value, and pointer to the record it relates to. This index structure is then sorted, allowing Binary Searches to be performed on it. Query plan: A query plan (or query execution plan) is an ordered set of steps used to access data in a SQL relational database management system. This is a specific case of the relational model concept of access plan. Database language: SQL includes all three basic database languages: Data definition language ("what is to be stored"). Data manipulation language ("how is the data changed when updates are made, "how we insert data", "how we delete data"). Query language. Database straitjackets: in order to handle complex data where attribute needs structure to store postgre object oriented rdbms model has been proposed. Active Component: events, actions such as trigger,stored procedure, conditions. MTTF: Mean time to failure (MTTF) is the length of time a device or other product is expected to last in operation. MTTF is one of many ways to evaluate the reliability of pieces of hardware or other technology. Big data example: Credit Card transaction information, Client information in large companies. Product sales information, Genetic information Environmental information, Stock-market information. Denial of service: make a network resource unavailable to its intended users for indefinitely longer period of time. Analytics: software handle big data are called analytics.
Slide_2:
we have three categories of objects in security: 1.Subjects (users, users' programs) 2 Objects (it is unclear what granularity of objects we select - it could be specific databases, specific tables, specific records, or even columns in records) 3. Privileges (read, write, update, possibly also *grant* (to implement delegation)). General access model: three-dimensional matrix, with Boolean entries 1.This matrix can be implemented in more than one way. For instance,implement it by means of *control access lists.* 2 Specifically, we can assign to an object a list consisting of records of the form (*subject*, *privilege*)3. Then, for instance, to see if *marek* can update the table *employees*, we check if the access list for the table *employees* includes an item (record): (*marek*, *update*). 3 This general access model *conceptually* relates to access lists,
discretionary access control: In this model the privileges are: select (i.e. read), write, update, drop, grant (actually, there are others)
1. When the user *marek* creates the table *clients* he is granted all privileges on that table 2. Thus *marek* can execute SQL queries on the table *Clients.3. maintain an access control table in the database system table. We can update this table by means of grant and revoke.*
Graph representation of permission: The graph *Gclients* is assigned to the table *clients* (Of course the construction is general, for each table we have a graph)*1.* The nodes are users; in our example *marek*, *dexter*, *alice*, *bob*, etc. 2.We add an additional node that we will call *system. 3.* The edges are always labeled with permissions and an additional bit denoting presence or absence of WITH GRANT OPTION 4.In our example, when *marek* created the table *clients*, we created edges from *system* to *marek. 5.* But edge in our graph is just a record: <h*grantor* , *grantee*, *privilege*, *grant bit* >. social engineering attack: Thus the following "social engineering" scenario is possible: the user *eve* tricks the user *dexter* into giving her permission to SELECT by arguing that it changes nothing. Indeed it does nothing at the time of grant. But later on, when *dexter* forgot about the fact that it was done, he gets a permission to SELECT with grant option from *marek* and *eve* gets the access to the table *clients* without *dexter* realizing the consequences of previous actions. Trojan horse attack: Once *eve* has a permission to SELECT, programs owned by her also have permissions to select (careful here!) 1. So here is what *eve* does. She breaks into *marek* account just once and since *marek* left his password and database password on a yellow "sticky note" at the back of his monitor, she connects to the database just once and executes the following command: GRANT SELECT ON *clients* TO *eve.* Now, *eve* writes a program that will copy the table *clients* every night at 2:00 a.m. to her account . 2. It is entirely legal from the point of view of DBMS
ccpccccccppobject or target. The Trusted Computer System Evaluation Criteria (TCSEC), the seminal work on the subject which is often referred to as the "Orange Book", defines MAC as "a means of restricting access to objects based on the sensitivity (as represented by a label) of the information contained in the objects and the formal authorization (i.e., clearance) of subjects to access information of such sensitivity". Also called "Bell – La Padula" model of security.
MAC model of security is based on two basic axioms: 1. Any user with classification c can write only to objects o that have classification t' such that $t \le t'$ 2. Any user with classification t can read only objects o that have classification t' such that $t' \le t$
Classifications of security levels: Unclassified < Confidential < Secret < Top Secret
Implementation: If the granularity is on the table level, we need two system tables: one for users, another for tables. If the granularity is on the record level, we need an additional attribute/field "security level" for each table. If the granularity is on the field level, then for each field in a table, we need an accompanying "security level" field.
B-LP axioms: The MAC model of security is based on the following two basic axioms:
1. "No write down": Any user with classification *t* can write only to objects *o* that have classification *t'* such that $t \le t$.
2. "No read up": Any user with classification *t* can read only objects *o* that have classification *t'* such that $t' \le t$.
Polyinstantiation: If the access control granularity is on the record or field level, then different users may see different answers to the same query. For granularity on the field level, this phenomenon leads to appearance of NULLs where we do not want to provide information.
Controlling derived information: need to control access to aggregates.
Compare with DAC:

DAC	MAC
Flexible, well suited for business applications	Tight, centralized
Supported by SQL	Not supported by SQL
Vulnerable to Trojan Horse attacks	Resistant to Trojan Horse attacks
No polyinstantiation	Problem of polyinstantiation
Easy to control subjects	Do not control enough of subjects

Other access control policies

Compartments: To get more control over the subjects that users can access, we can create many different compartments. Both the users and the tables are marked with compartment bits. A user can access a table which belonging to a certain compartment if his bit corresponding to that compartment is 1, otherwise he cannot access this table

This mechanism is neutral w.r.t. security model (you can have MAC, DAC or other models with compartments)

Biba model: A policy tailored towards preservation of data integrity (as opposed to secrecy) with axioms opposite to B-LP: No read down, no write up. This model is used as a standard for distribution of instructions, policies, etc. in all hierarchical organizations.

Role-Based Access Control (RBAC): Instead of granting privileges to individuals, we create roles and grant privileges to roles, and then assign individuals to different roles. In principle RBAC is neutral w.r.t. other access policies. We can have RBAC for DAC and RBAC for MAC. Supported by SQL (GRANT command can work on roles).

Chinese Wall policy (Brewer-Nash policy): Used in conflict-of-interest situations

The idea is that besides the objects (like in B-LP) we also have data sets, collections of objects. We assume that data sets form a partition of objects (think about collections of documents on specific clients). Then, we have a conflict relationship on the collection of data sets. This relation is supposed to be irreflexive and symmetric

Here are the rules: Access is granted to subject s on object o if either o is in a data set which s already accessed, or o is in a data set which is not in a data set which is in conflict with any of data sets accessed by s. Observe that there is no difference for reading or writing.

SQL Injection

An example:

```
myStr = "SELECT uid FROM users where uid = '" & userId & "' AND pwd = '" pass + "'"
result = GetQueryResult (myStr)
If (result = "") Then
auth = False
Else
auth = True
End If
```

Here userId and pass are taken from a Web form.

If I type as userId: ' OR ''='

And for pass: ' OR ''='

Funny thing happens.

Worse things may happen too because SQL allows for executing several commands at once. So, if within my string I put something like

; DROP TABLE accounts;

…

UNION was also used as a vehicle for SQL injection attacks.

Defending against SQL injection: All data needs to be sanitized of any string that should not be part of the input ▲ Never use string concatenation in SQL ▲ Prepared statements should be used as much as possible ▲ Application login should be implemented within well-validated stored procedure ▲ Encrypt, if the application is critical ▲ Use quotes on all parts of user input

Public key vs. private key: Private key encryption is the standard form. Both parties share an encryption key, and the encryption key is also the one used to decrypt the message.

Public key encryption uses two keys - one to encrypt, and one to decrypt. The sender asks the receiver for the encryption key, encrypts the message, and sends the encrypted message to the receiver. Only the receiver can then decrypt the message - even the sender cannot read the encrypted message.

Audit: General principle of access to databases: *There should be no unauthenticated access.* This is sometimes called C2 security, after US DoD book on security (the "Orange Book", from 1980s). The modern specifications of terminology and levels of security can be found in the document called "Common Criteria".

Basics: A basic requirement is that there must be audit trail for login to and logout from the DBMS. Generally, maintained as a table with time, name of the user, session ID, user name, which database is being used. It is easy to implement such audit trail with triggers, but specialized add-ons for DBMSes are available. The audit trail must be kept secure, possibly on another machine not directly accessible from the machine that has audited DBMS. What else needs to be audited depends on the type of applications.

Other aspects which may need auditing: ▲ How much each user uses the database, in terms of number of sessions, of space used, of programs that are being executed ▲ The time periods when the database is used (outside of "normal operating hours"?) ▲ Data-definition activity (copying of data that should not be kept longer than a specified period of time?) ▲ Accounts without passwords need to be purged. Quality of passwords must be audited ▲ Who created accounts? ▲ Database errors (can indicate SQL injection attacks) ▲ Changes to triggers and stored procedures ▲ Changes to privileges ▲ Changes to roles ▲ Usage of data that is deemed sensitive ▲ SELECT statements may need to be audited for privacy ▲ Dormant accounts are dangerous and so we need to audit them

Other notes: ▲ Without a well-established audit procedures DBMS is insecure ▲ DBA (or in larger places, IT department) is responsible for audit ▲ Audit is not a one-time activity but a continuing effort ▲ Auditing information should be properly archived because some auditing is required by law ▲ Use of encryption is beneficial because it proves persistence ▲ Auditing